2025 IEEE 36th International Symposium on Software Reliability Engineering Workshops (ISSREW) | 979-8-3315-5325-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISSREW67781.2025.00051

CECoder: Fine-Grained Code Element Retrieval for Repository-Level Code Generation

Yiming He
Intelligent Software Research Center
Institute of Software
Chinese Academy of Sciences
Nanjing Institute of Software Technology
University of Chinese Academy of Sciences
Nanjing, China
heyiming23@mails.ucas.ac.cn

Jingzheng Wu
Intelligent Software Research Center
Institute of Software
Chinese Academy of Sciences
Beijing, China
jingzheng08@iscas.ac.cn

Xiang Ling*
Intelligent Software Research Center
Institute of Software
Chinese Academy of Sciences
Beijing, China
lingxiang@iscas.ac.cn

Tianyue Luo
Intelligent Software Research Center
Institute of Software
Chinese Academy of Sciences
Beijing, China
tianyue@iscas.ac.cn

Mutian Yang
Beijing ZhongKeWeiLan
Technology Co.,Ltd.
Beijing, China
mutian@iscas.ac.cn

Chen Zhao
Intelligent Software Research Center
Institute of Software
Chinese Academy of Sciences
Beijing, China
zhaochen@iscas.ac.cn

***Abstract*—Large language models (LLMs) have demonstrated outstanding performance in standalone function-level code generation but still struggle with repository-level code generation due to limited understanding of the usage of the code elements (e.g., developer-defined classes and methods). Existing repository-level code generation methods adopt the retrieval-augmented generation (RAG) paradigm, employing whole-function semantic descriptions (e.g., task requirements) to retrieve code snippets assisting generation. However, this often leads to irrelevant context and subsequent code element misuse, resulting in logic errors, type mismatch, and other errors. To this end, we propose CECoder, a novel framework that performs fine-grained and accurate retrieval of concrete code element usage, demonstrating how elements are invoked and composed. In particular, CECoder first retrieve code snippets based on the task requirement, capturing whole-function semantic similarity to generate draft code. The draft code is then split into code blocks containing multiple code element invocations and preceding context, for which additional snippets are retrieved to reflect code element usage. All snippets are finally re-ranked and selectively integrated into a prompt template. To evaluate the effectiveness and generalization of CECoder, we conduct experiments on the widely used and representative DevEval benchmark, containing 1825 repository-level code generation tasks. CECoder consistently outperforms the state-of-the-art RepoCoder, across all evaluated metrics, achieving scores of 4.39%, 41.97%, 27.01%, and 39.25% in Exact Match, EditSimilarity, Pass@1, and Recall@1. It also generalizes across repositories of varying size and duplication.**

***Index Terms*—Code Generation, Large Language Model, Retrieval-Augmented Generation.**

I. Introduction

In recent years, large language models (LLMs) [1] [2] have demonstrated impressive capabilities in the field of code generation. However, their impressive performance has been observed in standalone function-level code generation, where generating the function-level code typically relies only on built-in or third-party libraries. In contrast, they face significant challenges in repository-level code generation that requires generating a complete and executable function body based on a provided task requirement, such as functional description and function signature, while appropriately integrating code elements (e.g., classes, methods) already defined within the target repository [3]. The difficulty arises from LLMs' limited knowledge of code elements [4] and their insufficient understanding of code element usages about how these elements invoke and compose to accomplish desired functionalities.

To address challenges in repository-level code generation, existing studies [5] [6] widely adopt the Retrieval-Augmented Generation (RAG) paradigm, retrieving code snippets from the target repository to guide code generation based on existing implementations. Some studies adopt single-round or iterative retrieval strategies, retrieving semantically similar snippets based on the task requirement or the generated code in the previous iteration [7]. These basic retrieval studies serve as the foundation for more advanced strategies: some studies incorporate structural or semantic information, such as the type context or dependency graphs, to build richer contexts [8] [9]. Others introduce adaptive optimization strategies, leveraging compiler results [10] or selective retrieval mechanism [11] to improve retrieval relevance. However, despite these research efforts, the core similarity-based retrieval paradigm remains limited. Specifically, it typically emphasizes whole-function semantic similarity based on task requirements or previously generated whole code while overlooking fine-grained code

* Xiang Ling is the corresponding author.

2994-810X/25/$31.00 ©2025 IEEE
DOI 10.1109/ISSREW67781.2025.00051

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:34:57 UTC from IEEE Xplore. Restrictio

element usage, which is crucial for the LLM to understand how to invoke and compose these code elements.

Motivated by it, we propose CECoder, a novel retrieval-augmented generation framework designed to exploit the diverse usages of code elements within the target repository and mitigate the possible misuses of code elements. CECoder first retrieves code snippets based on the task requirement and generates a draft code, outlining the target function's structural skeleton while incorporating potentially relevant code elements. Next, to construct query texts that focus on code element usages, CECoder splits the draft code into multiple code blocks, each containing a set of code element invocations and their preceding context. For each block, CECoder performs fine-grained retrieval to gather additional relevant snippets demonstrating diverse usages of code elements. Finally, CECoder applies a reciprocal rank fusion algorithm [12] to merge retrieval results from both whole-function semantic similarity and fine-grained code element usage, prioritizing snippets highly ranked or frequently retrieved across sources. This two-stage retrieval pipeline produces a context-rich prompt that better guides the LLM in generating executable code.

We conduct an evaluation of CECoder on the DevEval benchmark dataset [3], which features a code distribution that mirrors real-world repositories, thereby providing a more authentic evaluation of repository-level code generation. We evaluate CECoder using two popular LLMs: GPT-3.5-Turbo [1] and Code Llama [2]. Experimental results demonstrate that CECoder achieves 4.39%, 41.97%, 27.01%, and 39.25% in $\mathrm{ExactMatch}$, $\mathrm{EditSimilarity}$, $\mathrm{Pass@1}$, and $\mathrm{Recall@1}$, respectively, consistently outperforming the state-of-the-art RepoCoder [7]. Moveover, CECoder demonstrates strong generalizability, maintaining stable performance across repositories with diverse sizes and varying ratios of code duplication.

The main contributions of our work include:

- We propose CECoder, a novel retrieval-augmented code generation framework that introduces a two-phase retrieval strategy, combining whole-function semantic retrieval with fine-grained code element usage retrieval, to better support repository-level code generation.
- We design a code elements-aware retrieval mechanism that splits draft code into invocation-centered blocks and performs fine-grained retrieval to capture diverse usage of code elements.
- We conduct comprehensive experiments on the DevEval benchmark, demonstrating consistent better performance over baseline methods in both generation quality and generalization across repositories with diverse characteristics.

II. Related Work

Code generation typically takes place within code repositories, involving the understanding of various repository-specific code elements, which pose greater challenges to large language models (LLMs). Although LLMs demonstrate outstanding performance in standalone function-level code generation, they frequently produce unexecutable code when faced with repository-level contexts due to insufficient information [3].

To enhance the contextual understanding of LLMs, Retrieval-Augmented Generation (RAG) methods have been widely adopted. Typical studies such as RepoCoder [7] iteratively alternate between retrieval and generation, retrieving similar code snippets from the repository. RepoMinCoder [13] applies the minimum description length principle to measure information redundancy and loss. Some studies attempt to construct more precise retrieval contexts by parsing repository structure. A^3-CodGen [9] builds knowledge bases comprising local modules, global modules, and third-party libraries to enrich prompt representation; CATCODER [8] constructs type dependency graphs to extract accurate type contexts. Meanwhile, certain studies introduce feedback or selective retrieval mechanism to enable adaptive optimization. For example, CoCoGen [10] employs compiler error feedback to iteratively guide and refine the retrieval and generation process. Repoformer [11] introduces a selective retrieval mechanism to avoid unnecessary retrieval, thereby reducing redundant context.

However, few studies focus on retrieving code elements, especially their typical invocation usages, as CECoder does. By doing so, CECoder effectively enhances the accuracy and consistency of repository-level code generation, addressing common issues in existing studies such as incorrect code element usage and incomplete invocation combinations.

III. Methodology

In retrieval-based repository-level code generation, the objective is to generate a function body that is functionally correct and leverages existing code elements from the target code repository. Formally, each repository-level code generation task is defined by a task requirement $\mathcal{T} = \langle \text{signature}, \text{docstring} \rangle$, which consists of the function signature and the functional description in natural language. The target code repository is denoted as $\mathcal{S}_{repo} = \{s_1, s_2, \ldots, s_n\}$, representing a collection of available code snippets within the repository. The generation process is expressed as $\hat{y} = LLM(\mathcal{T}, \mathcal{S}_{refined})$, where $\hat{y}$ is the generated function body, and $\mathcal{S}_{refined} \subseteq \mathcal{S}_{repo}$ denotes a set of code snippets that contain the task-relevant code elements to support the generation.

Repository-level code generation is challenging for LLMs due to their limited understanding of repository-specific code elements and their usages. Existing retrieval methods often rely on whole-function semantic descriptions [7] [10], which lack sufficient signals to capture detailed code element usages.

To address this challenge, we propose CECoder, a retrieval-augmented generation framework designed to enhance LLMs' ability to integrate relevant code elements through fine-grained retrieval. The key to CECoder is how to retrieve a set of code snippets $\mathcal{S}_{refined}$ containing rich code element usages from $\mathcal{S}_{repo}$. An overview of the CECoder workflow is presented in Fig.1. Overall, CECoder consists of four stages: retrieval corpus construction, initial retrieval, code elements-aware retrieval, and merge. We detail the first two stages in SectionIII-A, and describe the code elements-aware retrieval and merging process in Sections III-B and III-C, respectively.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:34:57 UTC from IEEE Xplore. Restrictio

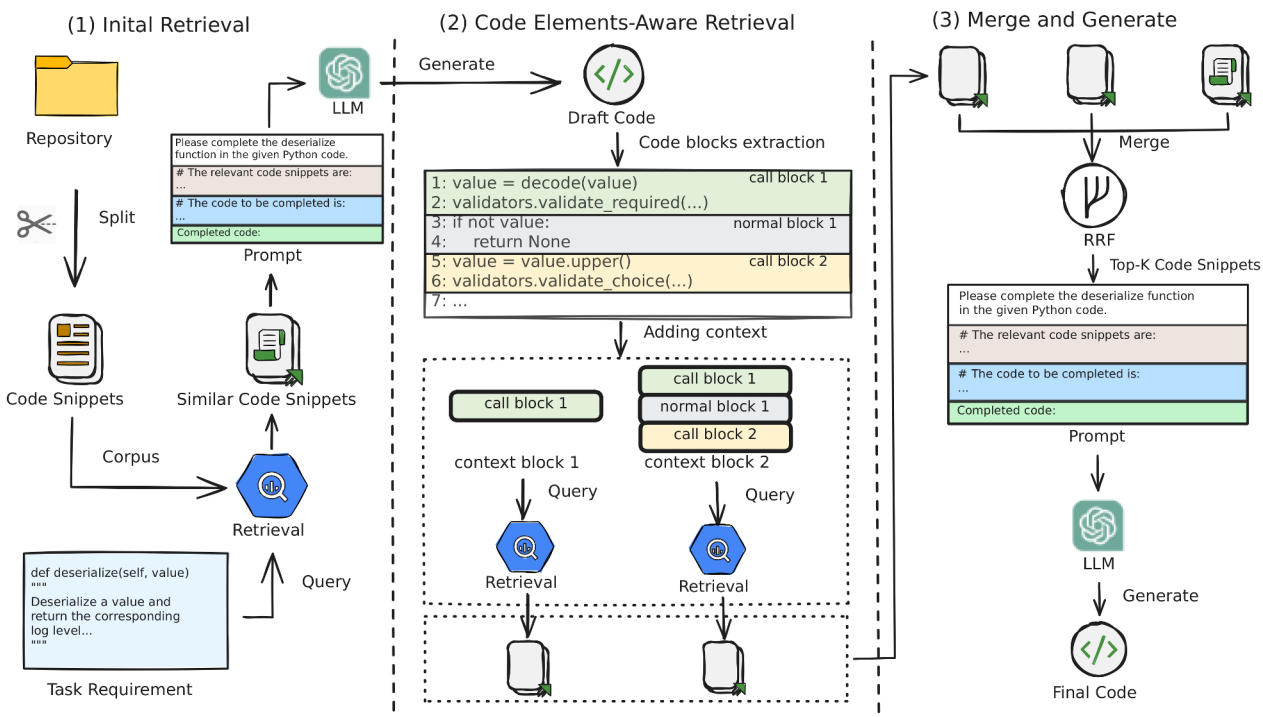

Fig. 1: Overview of the CECoder method, which leverages a two-stage retrieval process: first using the task requirement for initial whole-function semantic retrieval, followed by fine-grained retrieval of code element usages to refine the code generation process.

A. Initial Retrieval

Given a target function to be generated, its task requirement serves as a query text to retrieve similar code snippets from a retrieval corpus, whose construction from the target code repository will be detailed later. The retrieved snippets are organized and incorporated into a prompt to guide the LLM in generating a draft code.

Retrieval Corpus Construction. Since code repositories often contain many code files of varying and sometimes excessive lengths, using an entire file as the minimal retrieval unit is often suboptimal. To address this issue, we refer to the sliding window-based retrieval corpus construction strategy introduced in RepoCoder [7]. Specifically, we traverse all code files in the target repository and apply a sliding window of size $\mathcal{S}_w$ lines with a stride of $\mathcal{S}_s$ lines to scan the code of each file. Each windowed segment of code is extracted as a code snippet s_i $(i = 1, 2, \ldots, n)$. These segments are collected into a corpus of code snippets, denoted as $\mathcal{S}_{repo} = \{s_1, s_2, \ldots, s_n\}$, which serves as the retrieval corpus for subsequent stages.

Relevant Code Retrieval. Having constructed the retrieval corpus, we retrieve relevant code snippets based on the task requirement q, expressed as $\mathcal{S}_{initial} = R(\mathcal{S}_{repo}, q)$, where $R(\cdot)$ represents the retrieval mechanism applied to the corpus $\mathcal{S}_{repo}$. Specifically, given a query text q or a code snippet s_i, we convert each into a vector representation:

$$\mathbf{v}_q = \text{encoder}(q), \quad \mathbf{v}_{s_i} = \text{encoder}(s_i) \tag{1}$$

where encoder$(\cdot)$ denotes the text encoding function, which may be implemented as a sparse method (e.g., Bag-of-Words, TF-IDF) or a dense embedding model (e.g., Sentence-BERT, BGE). The resulting vectors $\mathbf{v}_q$ and $\mathbf{v}_{s_i}$ are used to calculate similarity between the query text and code snippets:

$$\text{score}_i = \text{sim}(\mathbf{v}_q, \mathbf{v}_{s_i}) \tag{2}$$

where sim$(\cdot, \cdot)$ denotes a similarity function, such as cosine similarity or Jaccard coefficient. The resulting score score$_i$ reflects the semantic similarity between the task requirement q and the code snippet s_i which is used to rank all code snippets in descending order. And then the top-K snippets with the highest scores are selected as the retrieval output set $\mathcal{S}_{initial}$.

Draft Code Generation. To address repository-leve code generation, we propose a new prompt template $\mathcal{P}$ based on the structure provided in DevEval [3]. The prompt consists of three essential components: task instruction, retrieved context, and task requirement. Specifically, we first insert the target function's name f_{name} into the task instruction $\mathcal{I}(f_{\text{name}})$. Next, we concatenate the top-K retrieved code snippets $\mathcal{S}_{initial}$, in ranked order to form the retrieval context $\mathcal{C}(\mathcal{S}_{initial})$. And then we concatenate the function signature and the functional description to create the task requirement $\mathcal{T}$:

$$\mathcal{P} = \mathcal{I}(f_{\text{name}}) + \mathcal{C}(\mathcal{S}_{initial}) + \mathcal{T} \tag{3}$$

Finally, the constructed prompt $\mathcal{P}$ is provided as input to the LLM, which is then tasked with generating the function body as a draft code. During the draft code generation, the LLM

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:34:57 UTC from IEEE Xplore. Restrictio

selects a set of code elements from the retrieved snippets that it deems relevant to the task and composes them to generate a code implementation that satisfies the task requirement.

B. Code Elements-Aware Retrieval

Although the draft code incorporates a set of code elements, the semantic gap between natural language and code may still lead to irrelevant code snippet retrieval, causing LLMs to misuse code elements. Therefore, directly using the draft code for final generation is suboptimal. Some methods [7] [10] [13] further perform iterative retrieval using the draft code itself. This code-to-code retrieval helps reduce irrelevant results to some extent based on the code elements contained in the draft code. However, we observe that some code snippets containing valuable usage of code elements may be overlooked when using the whole draft code as a single query text.

To address this, we propose splitting the draft into code blocks centered on code elements and using these blocks as separate query texts to retrieve code snippets similar to code element usage. Specifically, we parse the draft code into an abstract syntax tree (AST) and traverse the tree to locate all code element invocations. Consider that consecutive invocations may be semantically related and appear consecutively elsewhere in the code repository, consecutive invocations in the draft code are merged into a single block termed a call block. All other statements between call blocks are treated as a normal block, which typically contain variable declarations, control structures, or non-invocation logic. A call block represents a localized combination of code elements within the draft code and serves as a basis for retrieving code element usage in the repository. Based on the above definitions, the draft code $\mathcal{D}$ can be represented as a sequence of call blocks and normal blocks: $\mathcal{D} = [\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_n], \quad \mathcal{B}_i \in \{\mathcal{C}, \mathcal{N}\}$, where $\mathcal{C}$ and $\mathcal{N}$ denote the set of call blocks and normal blocks, respectively. $\mathcal{B}_i$ alternate between $\mathcal{C}$ and $\mathcal{N}$.

However, directly using isolated code blocks as query texts may disrupt the semantic integrity of the draft code. To address this, we construct a context block b_j for each call block $\mathcal{C}_j$ by concatenating all preceding blocks, including both call and normal blocks, with the current call block in their original order. This design is inspired by the common developer practice of envisioning potential element invocations based on existing logical implementations before retrieving relevant elements. Specifically, let $\mathcal{C}_j$ be located at position k in the draft sequence $\mathcal{D}$:

$$b_j = \text{concat}(\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_{k-1}, \mathcal{C}_j), \quad \mathcal{B}_i \in \{\mathcal{C}, \mathcal{N}\} \tag{4}$$

This formulation ensures that each context block ends with its corresponding call block $\mathcal{C}_j$, while the preceding blocks retain the structural logic necessary for effective retrieval. This design preserves the structural continuity of the original draft, allowing context blocks to retain relevant code dependencies.

After constructing a series of context blocks, we apply the same retrieval method as in Section III-A, using each context block as a query text to retrieve a set of top-K similar code snippets from the corpus of code snippets. The retrieval set $\mathcal{S}_{block_j}$ is an ordered list of code snippets, ranked by their similarity scores to the given context block b_j. Each retrieval set contains the usage of code elements in the target code repository that is similar to the corresponding context block.

C. Merge and Generate

After two-stage retrieval, we obtain multiple sets of code snippets. Due to the input length limitation of LLMs, it is infeasible to include all retrieved results directly in the prompt. To address this, we merge and rank the retrieved snippets to select the most relevant ones for constructing the final prompt.

While the code element-aware retrieval based on invocation-centered blocks captures fine-grained code element usage, the initial retrieval provides structural templates aligned with the whole-function semantic intent. Merging both enables the LLM to learn from complementary perspectives. Specifically, we employ the Reciprocal Rank Fusion (RRF) [8] [12] algorithm to merge the retrieved sets of code snippets. RRF is an algorithm that merges multiple result sets with different retrievals into a single unified set. This algorithm prioritizes snippets that rank high in multiple retrieved sets. It calculates an RRF score based on the ranking of snippets within each retrieved set, and if the same snippet appears in different sets, their RRF scores are summed to obtain the final score:

$$\text{RRF}(c) = \sum_{\mathcal{S} \in \mathcal{A}} \frac{1}{a + \text{rank}_{\mathcal{S}}(c)} \tag{5}$$

where $\mathcal{A}$ represents all retrieval result sets, represented as $\mathcal{A} = \{\mathcal{S}_{initial}, \mathcal{S}_{block_1}, \mathcal{S}_{block_2}, \ldots, \mathcal{S}_{block_m}\}$; $\text{rank}_{\mathcal{S}}(c)$ indicates the rank of the code snippet c in the retrieval set $\mathcal{S}$; a is a smoothing factor. After merging with RRF, we rank code snippets in descending order of their RRF scores and then select the top-K code snippets to form the refined retrieval set, denoted as $\mathcal{S}_{refined}$. Consistent with the prompt construction adopted during draft generation (Equation 3), we construct the final prompt using $\mathcal{S}_{refined}$, which is then used to guide the LLM in generating the final code.

IV. Experiment Settings

In this section, we introduce the benchmark datasets, evaluation metrics, baselines, and implementation details of our experiments. We formulate the following two research questions and answer them to demonstrate the effectiveness of CECoder.

- **RQ1(Overall Performance):** How effective is our method compared to other baseline methods in repository-level code generation?
- **RQ2(Generalizability):** Can our method generalize to different code repositories?

Datasets. Many benchmark datasets were not aligned with real-world code repositories when constructed, limiting the evaluation of LLMs' coding abilities in practical development scenarios [14] [15]. DevEval [3] addresses this by introducing 1825 repository-level code generation tasks from 117 high-quality repositories across 10 popular domains. It aligns with real-world repositories in terms of code distribution, dependency distribution, and scale, providing a more accurate

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:34:57 UTC from IEEE Xplore. Restrictio

measure of LLMs' actual coding abilities. Therefore, we adopt DevEval to evaluate the effectiveness of CECoder.

Evaluation Metrics. We adopt four complementary metrics to evaluate the generated code from exact match accuracy, lexical similarity, executable correctness and dependency coverage perspectives. $\mathrm{ExactMatch(EM)}$ measures whether the generated function exactly matches the reference implementation [15], offering a binary correctness signal. $\mathrm{EditSimilarity(ES)}$ captures lexical similarity between generated and ground truth by calculating normalized edit distance [7]. $\mathrm{Pass@k}$ evaluates executable correctness, reporting the proportion of generations that pass test cases within k samples [8] [10] [3]. $\mathrm{Recall@k}$ assesses dependency coverage by comparing invoked references with the ground truth [3]. We report results for k = 1, 5, and 10 for both $\mathrm{Pass@k}$ and $\mathrm{Recall@k}$.

Baselines. Although recent methods such as A^3-CodGen [9] and CoCoGen [10] incorporate additional structural information or adaptive optimization strategy, they fundamentally build upon typical retrieval-based methods. To benchmark CECoder's retrieval advancements appropriately, we select baselines that span the major retrieval paradigms used in prior methods, including ReACC [16], Vanilla RAG and RepoCoder [7]. ReACC combines sparse retrieval (BM25) with dense retrieval to enhance code snippet relevance. Vanilla RAG retrieves similar code snippets using the task requirement. RepoCoder iteratively retrieves code based on the code generated in the previous iteration. In addition, we include Direct Generation as a non-retrieval baseline, which inputs the task requirement into the LLM without any retrieved context.

Implementation Details. In this paper, we select GPT-3.5-Turbo (gpt-3.5-turbo-1106) and Code Llama (7B) as base models for evaluation, due to their frequent use in code generation benchmarks and to address performance and data leakage. In the Retrieval Corpus Construction, we segment the code repository using a sliding window of 500 lines with a stride of 50 lines, producing overlapping code snippets to ensure coverage and context preservation. In the Initial Retrieval and Code Elements-Aware Retrieval, we use the Bag-of-Words model to represent code snippets and calculate similarity via the Jaccard coefficient. The top-10 most similar snippets are selected. The above configuration ensures fair comparison with RepoCoder, though alternative retrieval and similarity functions can be applied. We set $a = 60$ in the RRF (Equation 5) as suggested by prior work [12] demonstrating its effectiveness in balancing ranking sensitivity and robustness. During code generation, we adhere to the practices outlined in DevEval [3], adopting two decoding strategies: (1) Greedy decoding, using temperature = 0 to generate a single solution; and (2) Nucleus sampling, with temperature = 0.4 and top-p = 0.95, generating 20 candidates per task. The maximum length of each generated code is 500 tokens.

V. Evaluation Results and Analysis

A. RQ1: Overall Performance

This research question evaluates the overall effectiveness of CECoder in repository-level code generation. We assess the performance of CECoder in comparison with Direction Generation and retrieval-augmented baselines, including single-round retrieval methods (e.g., ReACC and Vanilla RAG) and iterative retrieval methods (e.g., RepoCoder). Table I shows that CECoder achieves 27.01% $\mathrm{Pass@1}$, 39.25% $\mathrm{Recall@1}$, 4.39% $\mathrm{ExactMatch}$, and 41.97% $\mathrm{EditSimilarity}$, outperforming all baselines on the DevEval benchmark. Retrieval-augmented methods generally outperform Direct Generation due to the contextual information provided by retrieved code snippets. Iterative retrieval paradigms such as RepoCoder and CECoder surpass single-round methods by better capturing evolving generation contexts. However, RepoCoder's whole-function semantic retrieval may introduce irrelevant code snippets, particularly harming performance on smaller LLMs such as CodeLlama-7B, where it underperforms even single-round methods like Vanilla RAG. In contrast, CECoder mitigates this issue by retrieving code element usage.

TABLE I: PERFORMANCE COMPARISON OF METHODS

Method	EM	ES	Pass@1	Pass@5	Pass@10	Recall@1	Recall@5	Recall@10
LLM: GPT-3.5-Turbo								
Direct	0.33	31.26	14.19	17.76	19.31	15.27	17.37	18.27
ReACC	1.14	31.66	22.85	28.46	30.58	32.95	36.36	38.12
Vanilla RAG	2.26	34.87	25.53	30.23	31.84	35.73	39.92	41.16
RepoCoder	1.90	35.42	25.81	31.11	32.91	38.86	42.43	43.66
CECoder	**4.39**	**41.97**	**27.01**	**31.41**	**33.20**	**39.25**	**43.02**	**44.08**
LLM:Code Llama								
Direct	0.69	33.74	12.16	18.12	21.08	17.56	22.50	25.26
ReACC	2.06	36.28	12.88	19.53	23.14	22.57	31.15	35.80
Vanilla RAG	3.04	37.88	16.66	23.99	27.64	25.77	35.81	39.90
RepoCoder	2.75	37.48	16.44	23.53	27.05	25.33	34.92	38.49
CECoder	**3.43**	**38.19**	**17.37**	**24.69**	**28.32**	**26.31**	**36.78**	**39.91**

B. RQ2:Generalizability

To evaluate the generalizability of different methods, we analyze their performance across repositories with repository size and code duplication level. All experiments in this section are conducted using GPT-3.5-Turbo. For each of the 117 repositories in DevEval, we compute the lines of code (LOCs) and the code duplication rate, which is measured using `jscpd` as the ratio of duplicated lines to total LOCs. Repositories are then divided into four intervals based on each metric. For every interval, we report the $\mathrm{Pass@1}$ scores of all baselines to evaluate their performance.

1) Generalizability Across Different Repository Sizes: As shown in Fig. 2, all methods perform better on smaller repositories, where retrieved snippets can cover most or all of the repository. In contrast, for large repositories, retrieved snippets represent only a small portion of the repository. LLMs thus struggle to capture full context, making accurate retrieval crucial. CECoder achieves superior performance across all intervals, validating the effectiveness of its fine-grained code element usage retrieval.

2) Generalizability Across Different Duplication Rates: As shown in Fig. 3, CECoder maintains superior or competitive performance across all intervals, benefiting from fine-grained retrieval based on code element usage rather than relying on whole-function semantic similarity. Notably, duplication rate does not strictly correlate with $\mathrm{Pass@1}$. We hypothesize this is influenced by repository size: for example, repositories in

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:34:57 UTC from IEEE Xplore. Restrictio

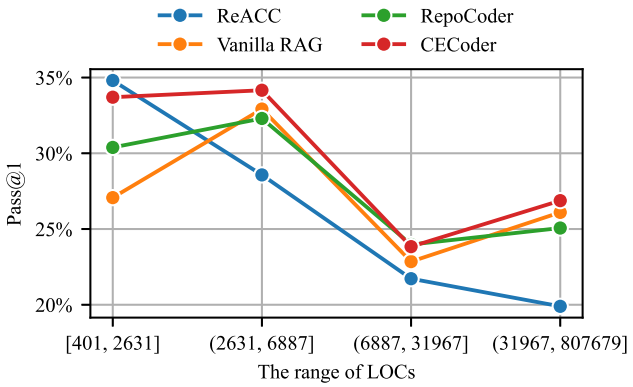

Fig. 2: Comparison of various methods at different code repository sizes

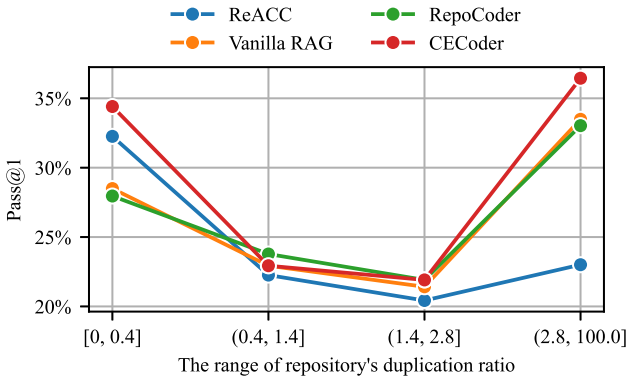

Fig. 3: Comparison of various methods in code repositories with different duplication rates

the [0, 0.4] interval have an average LOC of 3072.53, which, as shown in Fig. 2, falls into a range where LLMs generally achieve higher Pass@1 scores.

VI. CONCLUSION

This paper presents CECoder, a fine-grained retrieval framework that is aware of code element usage for repository-level code generation. CECoder employs a two-phase retrieval strategy: it first performs initial retrieval based on task requirement to obtain whole-function semantic code snippets, then conducts fine-grained retrieval guided by invocation-centered blocks to capture diverse code element usage. The integration of both retrieval results via reciprocal rank fusion enhances generation accuracy and reduces misuse of code elements. Experimental results on the DevEval benchmark demonstrate that CECoder consistently outperforms baselines across multiple metrics, achieving scores of 4.39%, 41.97%, 27.01%, and 39.25% in ExactMatch, EditSimilarity, Pass@1, and Recall@1, respectively. It also shows well generalization across repositories of different sizes and duplication levels.

ACKNOWLEDGMENTS

This paper is supported by the the Strategic Priority Research Program of the Chinese Academy of Sciences under No. XDA0320401, the National Natural Science Foundation of China under No. 62202457. This paper is also supported by YuanTu Large Research Infrastructure.

REFERENCES

[1] OpenAI, "ChatGPT," https://platform.openai.com/docs/models/gpt-3-5, 2023, accessed: 2025-03-18.

[2] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[3] J. Li, G. Li, Y. Zhao, Y. Li, H. Liu, H. Zhu *et al.*, "DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3603–3614.

[4] A. Eghbali and M. Pradel, "De-hallucinator: Mitigating llm hallucinations in code generation tasks via iterative grounding," *arXiv preprint arXiv:2401.01701*, 2024.

[5] D. Zan, B. Chen, Z. Lin, B. Guan, W. Yongji, and J.-G. Lou, "When Language Model Meets Private Library," in *Findings of the Association for Computational Linguistics: EMNLP 2022*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 277–288.

[6] M. Liu, T. Yang, Y. Lou, X. Du, Y. Wang, and X. Peng, "CodeGen4Libs: A Two-Stage Approach for Library-Oriented Code Generation," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Sep. 2023, pp. 434–445.

[7] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan *et al.*, "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 2471–2484.

[8] Z. Pan, X. Hu, X. Xia, and X. Yang, "Enhancing repository-level code generation with integrated contextual information," *arXiv preprint arXiv:2406.03283*, 2024.

[9] D. Liao, S. Pan, X. Sun, X. Ren, Q. Huang, Z. Xing *et al.*, "$\mathbf{A^3}$-codgen: A repository-level code generation framework for code reuse with local-aware, global-aware, and third-party-library-aware," *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3369–3384, 2024.

[10] Z. Bi, Y. Wan, Z. Wang, H. Zhang, B. Guan, F. Lu *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 2336–2353.

[11] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma, "Repoformer: Selective Retrieval for Repository-Level Code Completion," in *Proceedings of the 41st International Conference on Machine Learning*. PMLR, Jul. 2024, pp. 53 270–53 290.

[12] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. Boston MA USA: ACM, Jul. 2009, pp. 758–759.

[13] Y. Li, E. Shi, D. Zheng, K. Duan, J. Chen, and Y. Wang, "RepoMinCoder: Improving Repository-Level Code Generation Based on Information Loss Screening," in *Proceedings of the 15th Asia-Pacific Symposium on Internetware*. Macau China: ACM, Jul. 2024, pp. 229–238.

[14] H. Yu, B. Shen, D. Ran, J. Zhang, Q. Zhang, Y. Ma *et al.*, "CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, Feb. 2024, pp. 1–12.

[15] Y. Ding, Z. Wang, W. Ahmad, H. Ding, M. Tan, N. Jain *et al.*, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 46 701–46 723.

[16] S. Lu, N. Duan, H. Han, D. Guo, S.-w. Hwang, and A. Svyatkovskiy, "ReACC: A Retrieval-Augmented Code Completion Framework," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 6227–6240.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:34:57 UTC from IEEE Xplore. Restrictio